

```

1 |----- MODULE coordinationengine -----|
3 EXTENDS      FiniteSets, Sequences, Naturals, TLC
5 |-----|
7 The implementation of the CE should generate a sequence of agreements from submitted
8 proposals.
10 |-----|

13 VARIABLE    proposals,
14
15
16              rejections,
17              usedIds,
18              currentGsn,
19
20              agreements
21
22
23
24
25 CONSTANTS    Proposer,
26               ProposalId,
27
28               Reason,
29
30               Value,
31
32               MaxOutstandingProposals
33
34
35
36
37
38 ASSUME (MaxOutstandingProposals  $\in$  Nat)  $\wedge$  (MaxOutstandingProposals > 0)
39
40
41 |-----|
42 Tuple of all variables.
43
44
45 vars  $\triangleq$   $\langle$ proposals, rejections, usedIds, currentGsn, agreements $\rangle$ 
46
47
48 |-----|
49 Records.

```

```

52 Proposal  $\triangleq$  [type : "Proposal",
53             proposer : Proposer,
54             proposalId : ProposalId,
55             value : Value]

57 Agreement  $\triangleq$  [type : "Agreement",
58             proposer : Proposer,
59             proposalId : ProposalId,
60             value : Value,
61             agreementGsn : Nat]

63 Rejection  $\triangleq$  [type : "Rejection",
64             proposer : Proposer,
65             value : Value,
66             reason : Reason]

69 |-----|
70 | A function to return a proposal identity that hasn't been used before. This ensures each
71 | proposal has a unique identity. Also provided is a function to tell us if there are any proposal
72 | identities remaining, which is used as a constraint when model checking.

75 NewId  $\triangleq$  CHOOSE id  $\in$  ProposalId : id  $\notin$  usedIds

78 ProposalIdAvailable  $\triangleq$  Cardinality(usedIds) < Cardinality(ProposalId)

81 |-----|
82 | If the CE is accepting values, the proposal is submitted else the rejection is recorded.

85 Reject(p, v)  $\triangleq$  LET r  $\triangleq$  [type  $\mapsto$  "Rejection",
86                     proposer  $\mapsto$  p,
87                     value  $\mapsto$  v,
88                     reason  $\mapsto$  "NotAcceptingProposals"]
89                     IN  $\wedge$  rejections' = rejections  $\cup$  {r}
90                      $\wedge$  UNCHANGED  $\langle$ proposals, usedIds, currentGsn, agreements $\rangle$ 

93 SubmitProposal(p)  $\triangleq$   $\wedge$  proposals' = proposals  $\cup$  {p}
94                      $\wedge$  usedIds' = usedIds  $\cup$  {p.proposalId}
95                      $\wedge$  UNCHANGED  $\langle$ rejections, currentGsn, agreements $\rangle$ 

98 | There could be many reasons why proposals cannot be accepted, this is one example.

100 AcceptingProposals  $\triangleq$  Cardinality(proposals) < MaxOutstandingProposals

```

```

104 SubmitValue(p, v)  $\triangleq$  IF AcceptingProposals
105     THEN SubmitProposal([type  $\mapsto$  "Proposal",
106                          proposer  $\mapsto$  p,
107                          proposalId  $\mapsto$  NewId,
108                          value  $\mapsto$  v])
109     ELSE Reject(p, v)

112 |-----|
113 | To add an agreement to the sequence the proposal must be agreed. How a proposal is agreed is
114 | dependent on the consensus/coordination algorithm implemented by the CE.

117 AgreeProposal(p)  $\triangleq$  LET a  $\triangleq$  [type  $\mapsto$  "Agreement",
118                                proposer  $\mapsto$  p.proposer,
119                                proposalId  $\mapsto$  p.proposalId,
120                                value  $\mapsto$  p.value,
121                                agreementGsn  $\mapsto$  currentGsn + 1]
122     IN   $\wedge$  proposals' = proposals \ {p}
123          $\wedge$  currentGsn' = currentGsn + 1
124          $\wedge$  agreements' = Append(agreements, a)
125          $\wedge$  UNCHANGED  $\langle$ usedIds, rejections $\rangle$ 

128 EmitAgreement  $\triangleq$   $\exists p \in \text{proposals} : \text{AgreeProposal}(p)$ 

131 |-----|
132 | Type invariants.

135 TypeInvariant  $\triangleq$   $\wedge \forall id \in \text{usedIds} : id \in \text{ProposalId}$ 
136                    $\wedge \forall p \in \text{proposals} : p.type = \text{"Proposal"}$ 
137                    $\wedge \forall r \in \text{rejections} : r.type = \text{"Rejection"} \wedge r.reason \in \text{Reason}$ 
138                    $\wedge \text{Cardinality}(\text{proposals}) \leq \text{MaxOutstandingProposals}$ 
139                    $\wedge \text{currentGsn} \in \text{Nat}$ 

142 |-----|
143 | All agreed values must be an agreement, the proposalId must be in the set of usedIds (i.e., all
144 | agreements must be the result of a submitted proposal), the sequence number of each agreement
145 | must be in the correct position in the sequence.

148 CheckAgreement(a, i)  $\triangleq$   $\wedge a.type = \text{"Agreement"}$ 
149                    $\wedge a.proposalId \in \text{usedIds}$ 
150                    $\wedge a.agreementGsn = i$ 

153 CheckAgreementSequence[l  $\in$  Nat]  $\triangleq$ 

```

```

154                                     IF  $l = 0$ 
155                                     THEN TRUE
156                                     ELSE LET  $i \triangleq l - 1$ 
157                                     IN    $\wedge \text{CheckAgreement}(\text{agreements}[i], l)$ 
158                                      $\wedge \text{CheckAgreementSequence}[i]$ 

161  $\text{Safety} \triangleq$  LET  $l \triangleq \text{Len}(\text{agreements})$ 
162     IN    $\text{CheckAgreementSequence}[l]$ 

165 |-----|
166 | All submitted proposals should eventually be agreed.

169  $\text{Liveness} \triangleq \Diamond(\text{proposals} = \{\})$ 

172 |-----|
173 | The next state is found through either submitting a value or emitting an agreement.

176  $\text{Next} \triangleq \vee \exists p \in \text{Proposer} : (\exists v \in \text{Value} : \text{SubmitValue}(p, v))$ 
177      $\vee \text{EmitAgreement}$ 

180 |-----|
181 | Initial state.

184  $\text{Init} \triangleq \wedge \text{proposals} = \{\}$ 
185      $\wedge \text{rejections} = \{\}$ 
186      $\wedge \text{usedIds} = \{\}$ 
187      $\wedge \text{currentGsn} = 0$ 
188      $\wedge \text{agreements} = \langle \rangle$ 

191 |-----|
192  $\text{CESpec} \triangleq \wedge \text{Init}$ 
193      $\wedge \Box[\text{Next}]_{\text{vars}}$ 
194      $\wedge \text{Safety}$ 
195      $\wedge \text{Liveness}$ 
196 |-----|
197 THEOREM  $\text{CESpec} \Rightarrow \Box \text{TypeInvariant}$ 
198 |-----|

```